

Neural-Network Heuristics for Adaptive Bayesian Quantum Estimation

Lukas J. Fiderer^{1,2}, Jonas Schuff^{1,3}, Daniel Braun¹

¹*Eberhard-Karls-Universität Tübingen, Institut für Theoretische Physik, 72076 Tübingen, Germany*

²*School of Mathematical Sciences, The University of Nottingham,
University Park, Nottingham NG7 2RD, United Kingdom*

³*Department of Materials, University of Oxford, Parks Road, Oxford OX1 3PH, United Kingdom*

Quantum metrology promises unprecedented measurement precision but suffers in practice from the limited availability of resources such as the number of probes, their coherence time, or non-classical quantum states. The adaptive Bayesian approach to parameter estimation allows for an efficient use of resources thanks to adaptive experiment design. For its practical success fast numerical solutions for the Bayesian update and the adaptive experiment design are crucial. Here we show that neural networks can be trained to become fast and strong experiment-design heuristics using a combination of an evolutionary strategy and reinforcement learning. Neural-network heuristics are shown to outperform established heuristics for the technologically important example of frequency estimation of a qubit that suffers from dephasing. Our method of creating neural-network heuristics is very general and complements the well-studied sequential Monte-Carlo method for Bayesian updates to form a complete framework for adaptive Bayesian quantum estimation.

In quantum metrology we aim to design quantum experiments such that one or multiple parameters can be estimated from the measurement outcomes. Experiment design can involve the preparation of initial states, controlling the dynamics, or choosing measurements for readout. The estimation of parameters is a problem of statistical inference, and the most common approaches to tackle it are the frequentist and the Bayesian one.

In the frequentist approach, experiments are typically repeated several times which allows one to estimate the parameters from the statistic of measurement outcomes using, for example, maximum likelihood estimation. The problem of experiment design is often addressed with the Cramér–Rao bound formalism [1, 2] by maximizing the quantum Fisher information with respect to experiment designs [3].

The Bayesian approach, on the other hand, relies on updating the current knowledge about the parameters after each experiment using Bayes’ law. Examples for Bayesian quantum estimation involve state and process tomography [4–9], and phase and frequency estimation [10–12] with various experimental realizations [13–20]. The Bayesian approach is particular suitable for adaptive experiment design: experiments can be optimized depending on the current knowledge about the parameters and the available resources. While adaptivity can enhance the precision and save time and other resources compared to non-adaptive (frequentist) approaches [21, 22], it involves a computational challenge: The Bayesian update and the consecutive optimization of the experiment design are both analytically intractable (with rare exceptions under idealized conditions [10, 23]). In view of the short time scale of quantum experiments, slow numerical computation of the Bayesian update and the experiment design can drastically increase the total time consumed. In order to approximate the Bayesian update efficiently, a framework based on a

sequential Monte-Carlo (SMC) algorithm has been developed [6, 7, 24–26]. This framework, however, does not solve the problem of adaptive experiment design, which represents a second computational step.

In practice, one has to rely on so-called experiment-design heuristics, i.e., fast to evaluate functions which take available information as input and return an experiment design as output, see Fig. 1(a). If the output depends on the available information, we speak of an adaptive heuristic. So far, adaptive experiment-design heuristics for Bayesian estimation have been found mostly manually, typically motivated by analytic arguments derived for idealized conditions concerning the experimental model and the available resources [9–11, 19, 27]. In one case a manually found heuristic [27] has been fine-tuned offline using a particle swarm algorithm [28]. Another approach is to optimize experiment designs between the experiments with respect to a restricted set of experiment designs in order to keep the problem numerically tractable [29]. Apart from Bayesian inference, particle swarm and differential evolution algorithms have been used to search a certain class of experiment-design heuristics (represented by binary decision trees) in order to optimize the scaling of the uncertainty in phase estimation with the number of entangled photons in an interferometer [19, 30–32]. A general numerical framework for finding adaptive experiment-design heuristics for Bayesian quantum estimation is missing so far.

We consider an approach to experiment-design heuristics which uses reinforcement learning (RL). Recently, RL has been used with great success to create programs that play chess and Go better than any other program or human [33]. RL has also been used in quantum physics [34, 35] and, in particular, in quantum metrology such as for calibrating quantum sensors [36], for the identification light sources [37], and for improving the dynamics of quantum sensors [38, 39].

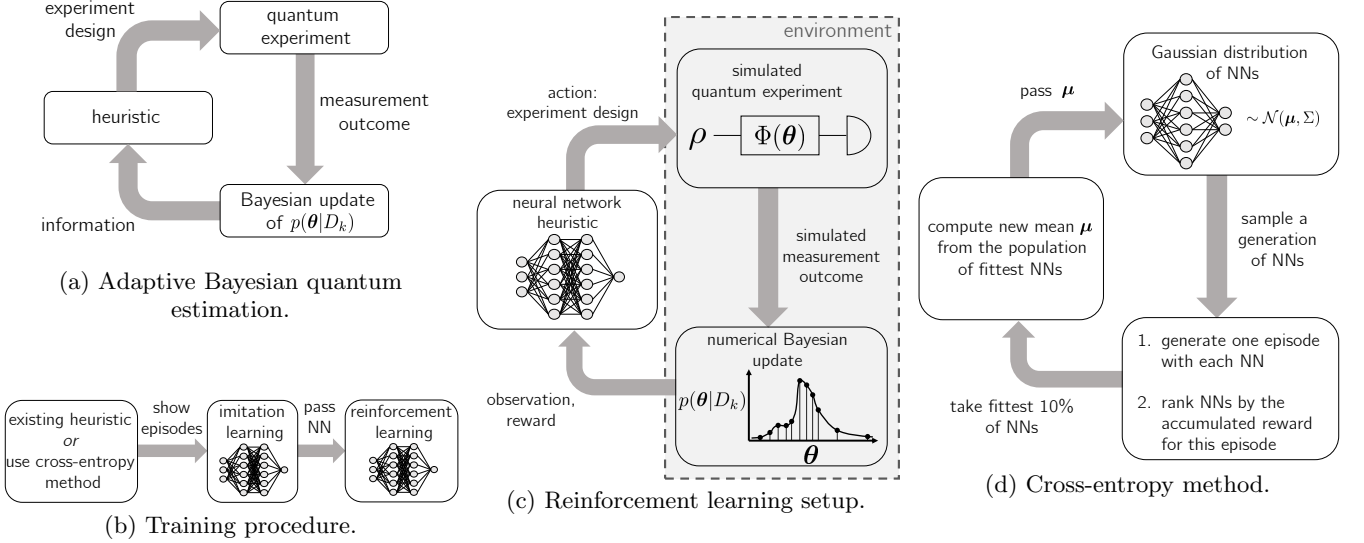


FIG. 1: Panel (a) shows the setup for adaptive Bayesian quantum estimation. First, prior information (e.g., about the prior $p(\theta)$ and the available resources) is passed to the heuristic which returns an experiment design e_1 . From the quantum experiment, we obtain a measurement outcome d_1 which is used to numerically compute the Bayesian update of $p(\theta)$. Then, it continues in the same manner and this procedure is repeated until some exit condition is fulfilled. Panel (b) depicts the training procedure of the neural networks (NNs). First, the NN learns to imitate the behavior of a known heuristic (which may have been found using the cross-entropy method). Then, we use this NN as a starting point for reinforcement learning (RL). Panel (c) shows the RL setup for generating training data. An agent (depicted as a NN) interacts with a RL environment. The RL environment defines the estimation problem. It consists of a simulated quantum experiment [depicted with initial state ρ and parameter-dependent quantum channel $\Phi(\theta)$] and a numerical Bayesian update. It also stores information about available resources for experiment design. Panel (d) shows the evolutionary principle of a simple cross-entropy method. NNs are treated as vectors of parameters sampled from a Gaussian distribution $\mathcal{N}(\mu, \Sigma)$ with constant covariance. At the beginning, the NNs are initialized randomly.

Here, we propose to use neural networks (NNs) as experiment-design heuristics. We provide a general method based on a combination of an evolutionary strategy with RL for creating such NN experiment-design heuristics. This method builds upon and compliments the SMC framework for approximate Bayesian updates [6, 7, 24–26]. It is general in the sense that (i) it can be easily adjusted to all kinds of estimation problems, (ii) it uses a very general ansatz for the experiment-design heuristic in form of a NN, and (iii) it creates a NN experiment-design heuristics that can take into account not only knowledge about the parameters to be estimated but also additional information, for instance, about available resources. Further, our method numerically approaches global optimization in the sense that it tries to find heuristics which are optimal for all future experiments as opposed to local (greedy) strategies which always optimize only the next experiment. The trained neural networks represent ready-to-use experiment-design heuristics and we envisage their application in Bayesian quantum sensors in combination with the SMC framework for approximate Bayesian updates [6, 7, 24–26].

BAYES RISK

Let $\theta = (\theta_1, \dots, \theta_d) \in \Theta$ be a vector of parameters we want to estimate, and we assume that Θ restricts each θ_j to a finite interval. The prior $p(\theta)$ is a probability distribution on Θ which represents our knowledge prior to the first measurement, and we imagine that prior to each Bayesian estimation θ is sampled from $p(\theta)$. After each experiment, our knowledge about θ is updated with Bayes' law (see Methods). Let $p(\theta|D_k, E_k)$ represent this updated knowledge about θ after the k th measurement, where $D_k = (d_1, \dots, d_k)$ denotes the measurement outcomes from a sequence of k experiments $E_k = (e_1, \dots, e_k)$. In the following, we omit the dependence on experiment designs for the sake of clarity, e.g., we write $p(\theta|D_k)$ instead of $p(\theta|D_k, E_k)$.

An experiment-design heuristic h is a function which maps available information, for instance about θ or available resources, to an experiment design for the next experiment, see Fig. 1 (a). The idea is to consult the experiment-design heuristic prior to each experiment and design the experiment accordingly. Imagine that the available resources for one Bayesian estimation are such

that we can make k experiments. Then, we aim to choose an experiment-design heuristic h which minimizes the expected traced covariance over $p(\boldsymbol{\theta}|D_k)$:

$$r[h|p(\boldsymbol{\theta})] = E_{D_k}(\text{tr}[\text{Cov}_{\boldsymbol{\theta}|D_k}(\boldsymbol{\theta})]), \quad (1)$$

where $\text{Cov}_{\boldsymbol{\theta}|D_k} = E_{\boldsymbol{\theta}|D_k}(\boldsymbol{\theta}\boldsymbol{\theta}^T) - E_{\boldsymbol{\theta}|D_k}(\boldsymbol{\theta})E_{\boldsymbol{\theta}|D_k}(\boldsymbol{\theta})^T$ is the covariance, and the notation $E_{a|b}(c)$ denotes the expected value of c with respect to $a \in A$ distributed as $p(a|b)$, $E_{a|b}(c) = \int_A da p(a|b)c$. If a takes discrete values the integral becomes a sum, and if a is distributed as $p(a)$ we write $E_a(c)$ instead of $E_{a|b}(c)$.

In Methods we show that $r[h|p(\boldsymbol{\theta})]$ corresponds to the Bayes risk for a loss function $L[\hat{\boldsymbol{\theta}}_k(D_k)|\boldsymbol{\theta}] = \|\hat{\boldsymbol{\theta}}_k(D_k) - \boldsymbol{\theta}\|_2^2$ with the Bayes estimator $\hat{\boldsymbol{\theta}}_k$ after k measurements given by $\hat{\boldsymbol{\theta}}_k(D_k) = E_{\boldsymbol{\theta}|D_k}(\boldsymbol{\theta})$. We also discuss in Methods how to generalize Eq. (1) if other resources are available, and how the expected values used for the computation of Eq. (1) are approximated numerically.

For a given estimation problem and prior $p(\boldsymbol{\theta})$, the Bayes risk $r[h|p(\boldsymbol{\theta})]$ represents our figure of merit (smaller values are better) for experiment-design heuristics.

EXPERIMENT DESIGN AS A RL PROBLEM

Our idea is to train a NN to become an experiment-design heuristic. To this end, we simulate the experiment and the Bayesian update offline many times to generate training data [see Fig. 1 (c)] and train the NN with it. Instead of simulating the experiment, measurement data from an actual experiment could be used to train the NN; note, however, that the calculation of Bayes update still depends on the model for the experiment. Either way, once the NN is trained, it provides a powerful adaptive heuristic that can be used as a part of a real Bayesian quantum sensor.

Let us phrase the problem of experiment-design heuristics in the language of RL. The neural network represents the RL agent. RL is an iterative method. In each iteration training data are generated and the agent learns from the data. The learning phase is prescribed by the RL algorithm and generally consists of a combination of learning from experience (using the training data) and random exploration.

The RL framework for generating training data is depicted in Fig. 1(c) and can be understood in the context of Bayesian quantum estimation, cf. Fig. 1(a): One *episode* of training data corresponds to one Bayesian estimation and consists of a sequence of k simulated experiments including Bayesian updates and experiment designs chosen by the RL agent. The number of experiments k depends on the available resources. In the simplest case, the number of experiments is the limiting resource (referred to as *experiment-limited* in the fol-

lowing), i.e., each episode consists of the same number of experiments, and the Bayes risk is given by Eq. (1). More generally, the available resources set more complicated constraints such that k can vary between different episodes (see Methods for a generalization of the Bayes risk for such cases). If a resource is exhausted, the *episode* ends, the RL environment [see Fig. 1(c)] is reset to default values (the current posterior is reset to the prior $p(\boldsymbol{\theta})$ and a new true parameter $\boldsymbol{\theta}$ is sampled from the prior), and another episode starts. Training data for one iteration typically consists of many episodes, e.g., $\sim 10^3$ in our model study below.

Crucial for the success of RL are the observations and rewards for the agent, see Fig. 1(c). The observations may contain information about the current knowledge $p(\boldsymbol{\theta}|D_k)$, about the past, such as prior actions, and about resources, such as the remaining time. The reward should reflect the goodness of the behavior (actions) of the agent (larger reward is better) and is used by the RL algorithm to enforce behavior which leads to larger rewards; the RL agent learns from its experience. The negative Bayes risk seems to be an obvious choice for a reward function. However, the computation of the Bayes risk is too time-consuming. Instead, we define the reward after the k th experiment as the difference in the traced covariance over the posterior after the k th and the $(k-1)$ th experiment,

$$R(D_k) = \text{tr}[\text{Cov}_{\boldsymbol{\theta}|D_{k-1}}(\boldsymbol{\theta})] - \text{tr}[\text{Cov}_{\boldsymbol{\theta}|D_k}(\boldsymbol{\theta})]. \quad (2)$$

The idea behind this reward function is that (i) Eq. (2) is straightforward to compute in the Bayesian SMC framework, (ii) it reflects the difference in our uncertainty about $\boldsymbol{\theta}$ before and after the current step, and (iii) the expected value of the rewards accumulated from the beginning of an episode with respect to possible measurement outcomes D_k yields the negative Bayes risk (up to a constant), see also Methods. For example, in the experiment-limited case with N experiments per episode we have

$$E_{D_N} \left[\sum_{j=1}^N R(D_j) \right] = \text{const} - r[h|p(\boldsymbol{\theta})], \quad (3)$$

where the constant is given by the uncertainty in the prior, $\text{const} = \text{tr}[\text{Cov}_{\boldsymbol{\theta}}(\boldsymbol{\theta})]$. RL has the goal to maximize the expected discounted reward which equals the left-hand side of Eq. (3) because we set the discount factor to one. From Eq. (3) we thus see that RL indeed attempts to minimize the Bayes risk $r[h|p(\boldsymbol{\theta})]$.

The training of the NNs consists of two steps, see Fig. 1 (b): We initialize the NN using imitation learning (pre-training) as implemented in [40]. The idea of imitation learning is to take advantage of an already existing (e.g., manually found) heuristic. The NN is trained to imitate the behavior from episodes created with the existing heuristic, cf. Fig. 1(c). This pretraining step is not strictly necessary but speeds up the training and makes RL more stable.

However, there might not always be a good heuristic available to imitate. For such cases and for the sake of comparison, we consider an evolutionary strategy [41] called the cross-entropy method (CEM) for continuous action spaces [42, 43] [see Fig. 1(d) and Methods]. CEM is used to train a neural network from scratch starting from a randomly initialized neural network.

Once the NN is pretrained, we use RL as a second step. The RL algorithm we use is trust region policy optimization (TRPO) [44] as implemented in the Python package Stable Baselines [40] (see Methods for details on the training and the advantages of combining it with CEM). TRPO is an approximation of an iterative procedure for optimizing policies with guaranteed monotonic improvement [44].

MODEL STUDY

We demonstrate our method of creating NN heuristics with an example of high practical relevance for magnetic field estimation with nitrogen-vacancy centers with applications in single-spin magnetic resonance [17, 18, 20, 45]. Let us consider a qubit which evolves under the Hamiltonian $H(\omega) = \frac{\omega}{2}\sigma_z$, and we want to estimate the frequency ω . The qubit is prepared in $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, evolves under $H(\omega)$ for a controllable time t , and is measured in the σ_x basis (assuming a strong projective measurement with outcomes labeled 0 and 1 corresponding to a measurement of $|+\rangle$ and $|-\rangle$). Let us further assume that the qubit suffers from an exponential decay of phase coherence, with characteristic time T_2 . According to the Born rule, the likelihood of finding an outcome $d \in \{0, 1\}$ with the σ_x measurement can be expressed as [7, 10],

$$p(0|\omega, t, T_2) = e^{-\frac{t}{T_2}} \cos^2\left(\frac{\omega}{2}t\right) + \frac{1 - e^{-\frac{t}{T_2}}}{2}, \quad (4)$$

for measuring $d = 0$, and $p(1|\omega, t, T_2) = 1 - p(0|\omega, t, T_2)$ for measuring $d = 1$. Eq. (4) defines all relevant properties of the experiment. A single experiment design consists of specifying the evolution time t . We consider the following estimation problems: (i) the estimation of ω without decoherence ($T_2 = \infty$, see the top row of panels in Fig. 2), (ii) the estimation of ω with known T_2 relaxation (we consider this problem twice with different values for T_2 , see the second and third row of panels in Fig. 2), and (iii) the simultaneous estimation of ω and T_2^{-1} , i.e., $\theta = (\omega, T_2^{-1})$ (see the bottom row of panels in Fig. 2). In all cases we consider $\omega \in (0, 1)$ (making the problem dimensionless).

Each estimation problem defines a RL environment [see Fig. 1(c)] which is either *time-limited* or *experiment-limited*. In the former case, the available time T per episode is limited, while the latter case the number of experiments N per episode is fixed. The first case is relevant if time is the limiting resource while the second case

is relevant if measurements are expensive, for instance, if experiments involve probing sensitive substances such as biological tissue. In practice, there may be constraints on both, T and N , which could be easily taken into account by creating a RL environment accordingly.

As an observation for the NN after the k th experiment we choose the expected value $E_{\theta|D_k}(\theta)$ and the covariance $\text{Cov}_{\theta|D_k}(\theta)$ over the posterior (that generalizes the variance in case of single-parameter estimation), the previous actions from the current episode (maximal 30 actions), and the spent time or the number of experiments in the current episode (for the time-limited or experiment-limited case, respectively).

Several heuristics have been developed for estimation problems (i) and (ii). As an example for a non-adaptive strategy, we consider a heuristic which chooses exponentially sparse times [10], $t_k = (9/8)^k$, denoted as *exp-sparse* heuristic in the following.

Further, we consider two adaptive heuristics: We define the first one as $t_k = \text{tr}[\text{Cov}_{\theta|D_{k-1}}(\theta)]^{-1/2}$ and we will call it σ^{-1} heuristic. This represents a generalization to multiparameter estimation of a heuristic which was derived for estimation problem (i) (ω estimation, $T_2 \rightarrow \infty$) by Ferrie *et al.* [10]. For single-parameter estimation, the σ^{-1} heuristic chooses the times t_k as the inverse standard deviation of θ over the posterior and is optimal in the greedy sense and only in the asymptotic limit $N \rightarrow \infty$.

The second adaptive heuristic that we consider is the *particle guess heuristic* (PGH) [11]. It is based on the SMC framework which uses a particle filter to represent probability distributions such as $p(\theta|D_k)$ [6] (see Methods). PGH chooses times as the inverse distance of two particles $\theta_1, \theta_2 \in \Theta$ sampled from $p(\theta|D_{k-1})$, $t_k = \|\theta_1 - \theta_2\|_2^{-1}$. In case of single-parameter estimation, PGH is a proxy for the σ^{-1} heuristic but it is faster to compute (given the particle filter) and introduces additional randomness (compared to the σ^{-1} heuristic).

Let us turn to the results depicted in Fig. 2. In all examples, we consider uniform priors for $\omega \in (0, 1)$ and, in case of multiparameter estimation, also for $T_2^{-1} \in (0.09, 0.11)$. For the multiparameter estimation examples, we consider, instead of one experiment, 100 independent and identical experiments in each step in order to facilitate the estimation of T_2^{-1} , and we also give the averaged outcome of these experiments as an additional observation to the RL agent.

We considered three different heuristics for pretraining TRPO: the σ^{-1} heuristic, PGH, and a CEM-trained NN, but plot only the results for the best of these three. For the data in Fig. 2, CEM is implemented with one hidden layer with 16 neurons, TRPO with a NN with two hidden layers with 64 neurons each. The NN heuristics outperform the conventional heuristics in all examples. Note that TRPO performs better than CEM when the limiting resource is used only partly. In the presence

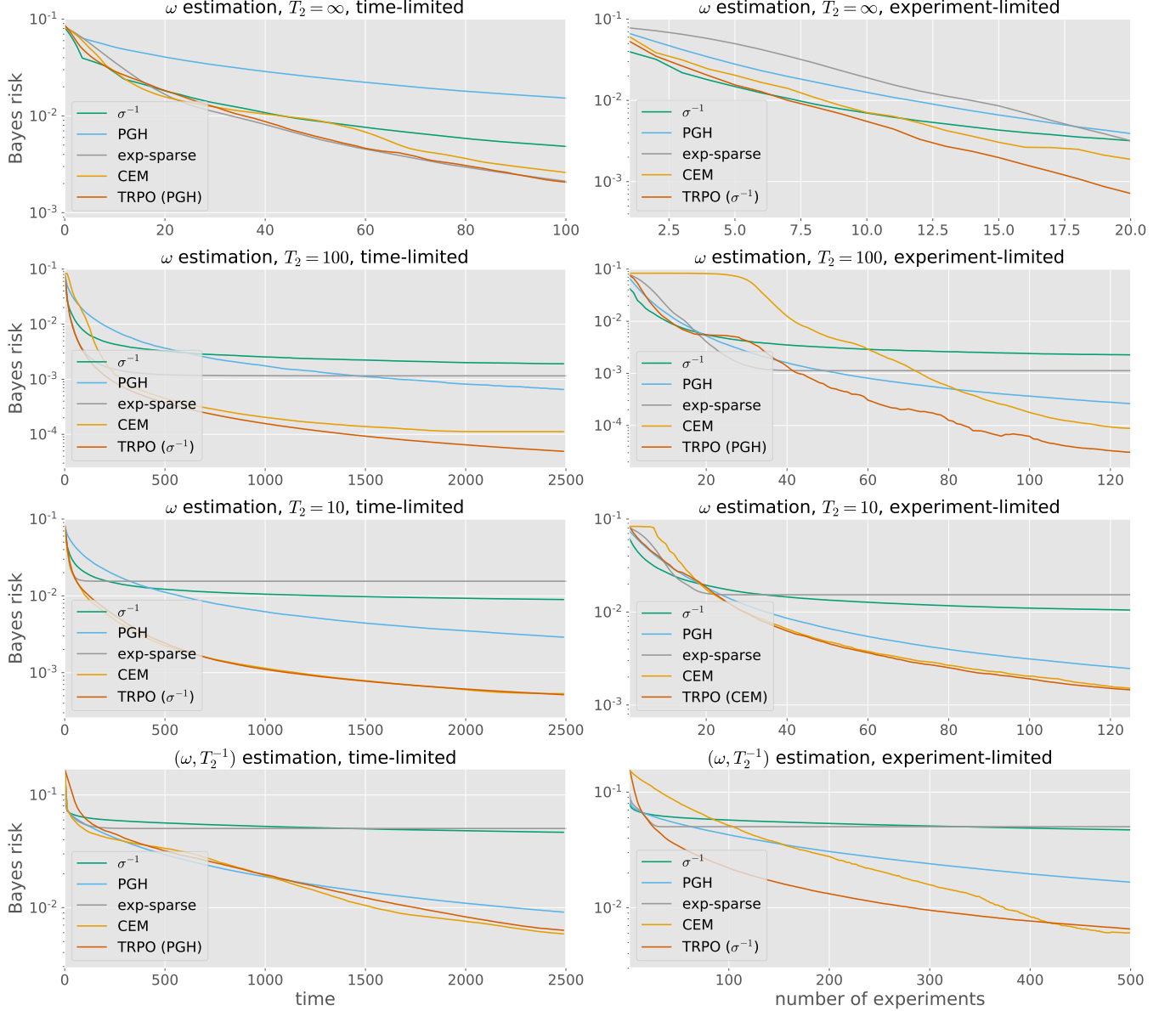


FIG. 2: Comparison of the Bayes risk for different experiment-design heuristics. The right column shows Bayesian estimation with a limited number of experiments, the left column with a limit on the available time (limited to the maximal value plotted on the x axis, respectively). We study frequency estimation without (top row) and with T_2 relaxation (2nd and 3rd row, with different values of T_2 as stated in the plot titles) as well as the simultaneous estimation of the frequency ω and relaxation rate T_2^{-1} (bottom row). The Bayes risk is calculated numerically from 10^4 episodes, see Methods for details. TRPO has been pretrained with the heuristic which is specified in brackets in the legends. The lines are linear interpolants to guide the eye.

of T_2 relaxation, times which exceed T_2 tend to yield no information, which explains why the Bayes risk saturates for the exp-sparse heuristic. The largest advantage of a NN heuristic is found for the example of time-limited ω estimation with $T_2 = 10$. Compared to PGH, we find an improvement in the Bayes risk by more than one order of magnitude. Generally, the performance of NN heuristics is remarkable given that (for the single-

parameter estimation problems considered in Fig. 2) the conventional heuristics such as PGH are used in experiments [17–20] and are considered to be the best practical choice with near-optimal performance [11, 12]. In the Supplementary Material, we compare the distribution of experiment-designs, i.e., the evolution times, for the different adaptive heuristics used in Fig. 2.

While the Bayes risk is used to compare the expected

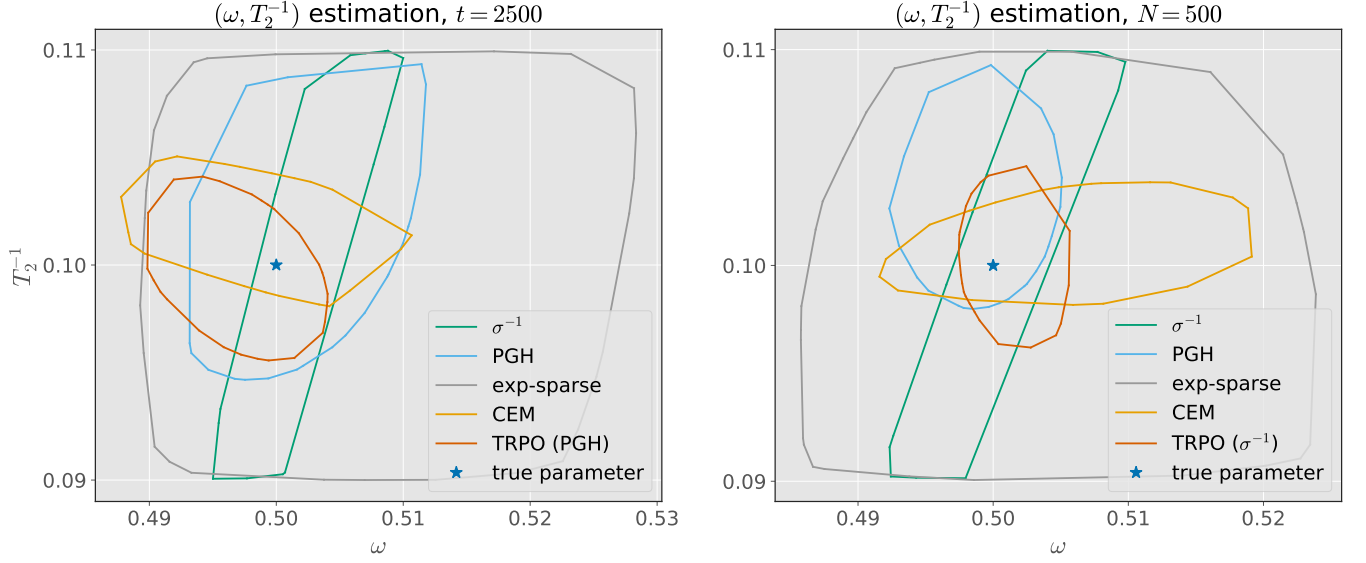


FIG. 3: 95% credible regions for multiparameter estimation of (ω, T_2^{-1}) . The credible regions correspond to the uncertainty in the posterior distributions obtained by running one Bayesian estimation (episode) for each heuristic while keeping the true parameter fixed. All heuristics use a uniform prior for $\omega \in (0, 1)$, $T_2^{-1} \in (0.09, 0.11)$.

(average) performance of heuristics, it is an important advantage of the Bayesian approach over the frequentist one that it provides credible regions as a practical tool for comparing single runs of parameter estimation (episodes) [25, 26]. Let us revisit the multiparameter problem discussed in the two bottom panels of Fig. 2. This time, we run only one Bayesian estimation of (ω, T_2^{-1}) with each heuristic and visualize their performance by plotting 95% credible regions [25, 26], see Fig. 3. Note that since the Bayesian estimation is subject to fluctuations (such as stochastic measurement outcomes), the shape of credible regions fluctuates between different estimations. For the specific example shown, we see that TRPO provides the smallest uncertainties compared to the other heuristics as judged by the area of the credible regions.

DISCUSSION AND CONCLUSION

The practical success of adaptive Bayesian estimation often depends on the run time of data processing (Bayesian update, choice of an adaptive experiment design). Using NNs as experiment-design heuristics introduces a computation overhead compared to the fastest existing experiment-design heuristics such as PGH. On the other hand, improvements in measurement precision achieved by the NN heuristics may easily outweigh the drawback of an increased run time. This trade-off has to be assessed dependent on the estimation problem and its concrete realization. In our model study, the run time for one call to a NN heuristic is always shorter than the corresponding numerical Bayesian update (single-core com-

putation in both cases). For more complicated modeling of the sensor or for an estimation up to a larger precision, the run time of the Bayesian update increases even more and we expect that the run time will be dominated by the numerical Bayesian update. Moreover, the effective run time of the NN per experiment could be reduced by using smaller NNs or by sacrificing a part of the adaptivity, e.g., by calling the NN heuristics only every m th experiment in which case the NN could return the designs for the next m experiments.

In conclusion, we proposed and demonstrated a machine learning method to create fast and strong experiment-design heuristics for Bayesian quantum estimation. The method uses imitation and reinforcement learning for training NNs to become experiment-design heuristics. In order to make the method independent of the availability of an expert heuristic for imitation learning, we show that expert heuristics can be found with an evolutionary strategy (the cross-entropy method). The big advantage of our method is its versatility and adaptivity. The properties of the estimation problem and the quantum experiments, and the availability of resources are taken into account during the training such that the trained NNs are tailored experiment-design heuristics. Similarly, we expect that issues such as uncertainty in the model or multimodal probability distributions are automatically taken into account by NN heuristics as a result of training. We provide the complete source code [46] used for this work in order to facilitate the application of the presented method in experiments and to related problems such as the detection of time-dependent signals [25, 47] and adaptive Bayesian state tomography [8, 9].

L.F. and D.B. acknowledge support from the Deutsche Forschungsgemeinschaft (DFG), Grant No. BR 5221/1-1. L.F. thanks Paul Knott for helpful discussions about RL.

-
- [1] C. W. Helstrom, *Quantum Detection and Estimation Theory*, Mathematics in Science and Engineering, Vol. 123 (Elsevier, 1976).
 - [2] S. L. Braunstein and C. M. Caves, Statistical distance and the geometry of quantum states, *Phys. Rev. Lett.* **72**, 3439 (1994).
 - [3] L. J. Fiderer, J. M. E. Fraïsse, and D. Braun, Maximal Quantum Fisher Information for Mixed States, *Phys. Rev. Lett.* **123**, 250502 (2019).
 - [4] V. Bužek, R. Derka, G. Adam, and P. Knight, Reconstruction of Quantum States of Spin Systems: From Quantum Bayesian Inference to Quantum Tomography, *Annals of Physics* **266**, 454 (1998).
 - [5] R. Blume-Kohout, Optimal, reliable estimation of quantum states, *New Journal of Physics* **12**, 043034 (2010).
 - [6] F. Huszár and N. M. T. Hounsby, Adaptive Bayesian quantum tomography, *Phys. Rev. A* **85**, 052120 (2012).
 - [7] C. E. Granade, C. Ferrie, N. Wiebe, and D. G. Cory, Robust online Hamiltonian learning, *New Journal of Physics* **14**, 103013 (2012).
 - [8] C. Granade, J. Combes, and D. G. Cory, Practical Bayesian tomography, *New Journal of Physics* **18**, 033024 (2016).
 - [9] C. Granade, C. Ferrie, and S. T. Flammia, Practical adaptive quantum tomography, *New Journal of Physics* **19**, 113017 (2017).
 - [10] C. Ferrie, C. E. Granade, and D. G. Cory, How to best sample a periodic probability distribution, or on the accuracy of Hamiltonian finding strategies, *Quantum Information Processing* **12**, 611 (2012).
 - [11] N. Wiebe, C. Granade, C. Ferrie, and D. Cory, Hamiltonian Learning and Certification Using Quantum Resources, *Phys. Rev. Lett.* **112**, 190501 (2014).
 - [12] N. Wiebe and C. Granade, Efficient Bayesian Phase Estimation, *Phys. Rev. Lett.* **117**, 010503 (2016).
 - [13] K. S. Kravtsov, S. S. Straupe, I. V. Radchenko, N. M. T. Hounsby, F. Huszár, and S. P. Kulik, Experimental adaptive Bayesian tomography, *Phys. Rev. A* **87**, 062122 (2013).
 - [14] G. I. Struchalin, I. A. Pogorelov, S. S. Straupe, K. S. Kravtsov, I. V. Radchenko, and S. P. Kulik, Experimental adaptive quantum tomography of two-qubit states, *Phys. Rev. A* **93**, 012103 (2016).
 - [15] R. J. Chapman, C. Ferrie, and A. Peruzzo, Experimental Demonstration of Self-Guided Quantum Tomography, *Phys. Rev. Lett.* **117**, 040402 (2016).
 - [16] Z. Hou, J.-F. Tang, C. Ferrie, G.-Y. Xiang, C.-F. Li, and G.-C. Guo, Experimental realization of self-guided quantum process tomography, *arXiv preprint arXiv:1908.01082* (2019).
 - [17] J. Wang, S. Paesani, R. Santagati, S. Knauer, A. A. Gentile, N. Wiebe, M. Petruzzella, J. L. O'Brien, J. G. Rarity, A. Laing, and M. G. Thompson, Experimental quantum Hamiltonian learning, *Nature Physics* **13**, 551 (2017).
 - [18] S. Paesani, A. Gentile, R. Santagati, J. Wang, N. Wiebe, D. Tew, J. O'Brien, and M. Thompson, Experimental Bayesian Quantum Phase Estimation on a Silicon Photonic Chip, *Phys. Rev. Lett.* **118**, 100503 (2017).
 - [19] A. Lumino, E. Polino, A. S. Rab, G. Milani, N. Spagnolo, N. Wiebe, and F. Sciarrino, Experimental Phase Estimation Enhanced by Machine Learning, *Phys. Rev. Appl.* **10**, 044033 (2018).
 - [20] R. Santagati, A. Gentile, S. Knauer, S. Schmitt, S. Paesani, C. Granade, N. Wiebe, C. Osterkamp, L. McGuinness, J. Wang, M. Thompson, J. Rarity, F. Jelezko, and A. Laing, Magnetic-Field Learning Using a Single Electronic Spin in Diamond with One-Photon Readout at Room Temperature, *Phys. Rev. X* **9**, 021019 (2019).
 - [21] K. Chaloner and I. Verdinelli, Bayesian experimental design: A review, *Statistical Science* **10**, 273 (1995).
 - [22] A. Sergeevich, A. Chandran, J. Combes, S. D. Bartlett, and H. M. Wiseman, Characterization of a qubit Hamiltonian using adaptive measurements in a fixed basis, *Phys. Rev. A* **84**, 052315 (2011).
 - [23] A. M. Childs, J. Preskill, and J. Renes, Quantum information and precision measurement, *Journal of Modern Optics* **47**, 155 (2000).
 - [24] J. Liu and M. West, Combined Parameter and State Estimation in Simulation-Based Filtering, in *Sequential Monte Carlo Methods in Practice* (Springer New York, 2001) pp. 197–223.
 - [25] C. Granade, C. Ferrie, I. Hincks, S. Casagrande, T. Alexander, J. Gross, M. Kononenko, and Y. Sanders, QInfer: Statistical inference software for quantum applications, *Quantum* **1**, 5 (2017).
 - [26] C. Granade, C. Ferrie, S. Casagrande, I. Hincks, M. Kononenko, T. Alexander, and Y. Sanders, QInfer: Library for Statistical Inference in Quantum Information (2016).
 - [27] M. P. Stenbergh, Y. R. Sanders, and F. K. Wilhelm, Efficient estimation of resonant coupling between quantum systems, *Phys. Rev. Lett.* **113**, 210404 (2014).
 - [28] M. P. V. Stenbergh, O. Köhn, and F. K. Wilhelm, Characterization of decohering quantum systems: Machine learning approach, *Phys. Rev. A* **93**, 012122 (2016).
 - [29] I. Hincks, T. Alexander, M. Kononenko, B. Soloway, and D. G. Cory, Hamiltonian Learning with Online Bayesian Experiment Design in Practice, *arXiv preprint arXiv:1806.02427* (2018).
 - [30] A. Hentschel and B. C. Sanders, Machine Learning for Precise Quantum Measurement, *Phys. Rev. Lett.* **104**, 063603 (2010).
 - [31] A. Hentschel and B. C. Sanders, Efficient Algorithm for Optimizing Adaptive Quantum Metrology Processes, *Phys. Rev. Lett.* **107**, 233601 (2011).
 - [32] N. B. Lovett, C. Crosnier, M. Perarnau-Llobet, and B. C. Sanders, Differential Evolution for Many-Particle Adaptive Quantum Metrology, *Phys. Rev. Lett.* **110**, 220501 (2013).
 - [33] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, *et al.*, A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play, *Science* **362**, 1140 (2018).
 - [34] P. Palittapongarnpim, P. Wittek, E. Zahedinejad, S. Vedaie, and B. C. Sanders, Learning in quantum control: High-dimensional global optimization for noisy quantum dynamics, *Neurocomputing* **268**, 116 (2017).
 - [35] V. Dunjko and H. J. Briegel, Machine learning & artificial intelligence in the quantum domain: a review of re-

- cent progress, Reports on Progress in Physics **81**, 074001 (2018).
- [36] V. Cimini, I. Gianani, N. Spagnolo, F. Leccese, F. Sciarino, and M. Barbieri, Calibration of Quantum Sensors by Neural Networks, Phys. Rev. Lett. **123**, 230502 (2019).
 - [37] C. You, N. Bhusal, A. Lambert, C. Dong, A. Perez-Leija, R. d. J. Leon-Montiel, A. Javaid, and O. S. Magana-Loaiza, Identification of light sources using artificial neural networks, arXiv preprint arXiv:1909.08060 (2019).
 - [38] H. Xu, J. Li, L. Liu, Y. Wang, H. Yuan, and X. Wang, Transferable control for quantum parameter estimation through reinforcement learning, arXiv preprint arXiv:1904.11298 (2019).
 - [39] J. Schuff, L. J. Fiderer, and D. Braun, Improving the dynamics of quantum sensors with reinforcement learning, New Journal of Physics **10.1088/1367-2630/ab6f1f** (2020).
 - [40] A. Hill, A. Raffin, M. Ernestus, A. Gleave, A. Kanervisto, R. Traore, P. Dhariwal, C. Hesse, O. Klimov, A. Nichol, M. Plappert, A. Radford, J. Schulman, S. Sidor, and Y. Wu, Stable Baselines, <https://github.com/hill-a/stable-baselines> (2018).
 - [41] P. A. Knott, A search algorithm for quantum state engineering and metrology, New Journal of Physics **18**, 073033 (2016).
 - [42] R. Rubinstein, The cross-entropy method for combinatorial and continuous optimization, Methodology and computing in applied probability **1**, 127 (1999).
 - [43] A. Pourchot and O. Sigaud, CEM-RL: Combining evolutionary and gradient-based methods for policy search, arXiv preprint arXiv:1810.01222 (2018).
 - [44] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, Trust region policy optimization, in *International conference on machine learning* (2015) pp. 1889–1897.
 - [45] S. Schmitt, T. Gefen, F. M. Strner, T. Unden, G. Wolff, C. Mller, J. Scheuer, B. Naydenov, M. Markham, S. Pezzagna, J. Meijer, I. Schwarz, M. Plenio, A. Retzker, L. P. McGuinness, and F. Jelezko, Submillihertz magnetic spectroscopy performed with a nanoscale quantum sensor, Science **356**, 832 (2017).
 - [46] Source code and data are available at <https://doi.org/10.6084/m9.figshare.11927043>.
 - [47] M. Isard and A. Blake, Condensation—conditional density propagation for visual tracking, International journal of computer vision **29**, 5 (1998).
 - [48] A. Doucet and A. M. Johansen, A tutorial on particle filtering and smoothing: Fifteen years later, Handbook of nonlinear filtering **12**, 3 (2009).
 - [49] S. van der Walt, S. C. Colbert, and G. Varoquaux, The NumPy Array: A Structure for Efficient Numerical Computation, Computing in Science Engineering **13**, 22 (2011).
 - [50] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, OpenAI Gym (2016), arXiv:1606.01540.
 - [51] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, PyTorch: An imperative style, high-performance deep learning library, in *Advances in Neural Information Processing Systems* (2019) pp. 8024–8035.
 - [52] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems (2015), software available from tensorflow.org.
 - [53] M. A. Nielsen, *Neural networks and Deep Learning*, Vol. 25 (Determination press San Francisco, CA, USA, 2015).
 - [54] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, arXiv preprint arXiv:1412.6980 (2014).

METHODS

Bayes' law and the sequential Monte-Carlo algorithm

Bayes' law for updating our knowledge about $\theta \in \Theta$ according to the measurement outcome d_k of the k th experiment is given by

$$p(\theta|D_k) = \frac{p(d_k|\theta)p(\theta|D_{k-1})}{p(d_k)}, \quad (5)$$

where $p(\theta|D_k)$ is our updated knowledge (posterior), $p(\theta|D_{k-1})$ is our knowledge prior to the k th experiment, $p(d_k|\theta)$ is the likelihood of measuring d_k , and $p(d_k)$ is a normalization, $p(d_k) = \mathbb{E}_\theta[p(d_k|\theta)]$. The exact solution for the Bayesian update is generally intractable. Instead, we use an inference algorithm based on the sequential Monte-Carlo algorithm [6, 7, 48]. The idea is to represent the probability distributions $p(\theta)$ and $p(\theta|D_k)$ by a discrete approximation $\sum_{j=1}^n w_j \delta(\theta - \theta_j)$ with n so-called particles with positive weights w_j and positions $\theta_j \in \Theta$. Then, for a Bayesian update, we only need to update the weights of each particle by calculating $p(d_k|\theta_j)$. This means that for the j th particle we have to simulate the experiment for $\theta = \theta_j$ and use the Born rule to find $p(d_k|\theta_j)$. The expected value in the definition of $p(d_k)$ reduces to a simple sum over the particles of the prior.

The particle locations need to be resampled if too many weights are close to zero, i.e., the particle filter of $p(\theta|D_k)$ is impoverished. We use Qinfer's [26] implementation of the Liu–West resampling algorithm [24] (with default parameter $a = 0.98$ [26]) with $n = 2 \times 10^3$ particles for RL environments without decoherence and for RL environments with multiparameter estimation. For the environments with ω estimation and finite T_2 we use $n = 2 \times 10^4$ particles.

Bayes risk

Let us consider a sequence of k experiments designed with the experiment-design heuristic h . Let $\hat{\theta}_k : D_k \mapsto \hat{\theta}_k(D_k)$ be an estimator of θ after k experiments, and let $L[\hat{\theta}_k(D_k)|\theta]$ be a loss function which quantifies the deviation of $\hat{\theta}_k(D_k)$ from θ . For an experiment-design heuristic h , the risk s of $\hat{\theta}_k$ is defined as (usually denoted by R which denotes the reward in this work)

$$s(\hat{\theta}_k, h|\theta) = \mathbb{E}_{D_k|\theta} [L[\hat{\theta}_k(D_k)|\theta]]. \quad (6)$$

Note that the experiment-design heuristic h determines the experiment designs E_k which influence the estimate $\hat{\theta}_k(D_k, E_k)$. However, in order to simplify notation we write $\hat{\theta}_k(D_k)$ instead of $\hat{\theta}_k(D_k, E_k)$. For a definition of the expected value $\mathbb{E}_{D_k|\theta}$, see in the main text after Eq. (1). Eq. (6) is the risk typically associated with

a choice of an estimator $\hat{\theta}_k$ and corresponds to the expected (with respect to measurement outcomes D_k) loss. The Bayes risk represents a way to additionally take into account prior knowledge about θ in form of a probability distribution $p(\theta)$ (the prior) on Θ . Given a prior $p(\theta)$, the Bayes risk is defined as

$$r[\hat{\theta}_k, h|p(\theta)] = \mathbb{E}_\theta [s(\hat{\theta}_k, h|\theta)]. \quad (7)$$

A common choice for a loss function is the quadratic loss $L[\hat{\theta}_k(D_k)|\theta] = \|\hat{\theta}_k(D_k) - \theta\|_2^2$ [7]. Then, the estimator which minimizes the Bayes risk, the Bayes estimator, is given by the expectation over the posterior, $\hat{\theta}_k(D_k) = \mathbb{E}_{\theta|D_k}(\theta)$. For the quadratic loss, the risk is also known as mean squared error, and the Bayes estimator is also known as minimum mean square error estimator.

Derivation of Eq. (1) in the main text

Let us show that Eq. (7) is equivalent to the Bayes risk defined in Eq. (1) in the main text. We find from Eq. (7),

$$r[\hat{\theta}_k, h|p(\theta)] = \mathbb{E}_\theta \left[\mathbb{E}_{D_k|\theta} \left(\|\hat{\theta}_k(D_k) - \theta\|_2^2 \right) \right] \quad (8)$$

$$= \mathbb{E}_{D_k} \left[\mathbb{E}_{\theta|D_k} \left(\|\hat{\theta}_k(D_k) - \theta\|_2^2 \right) \right], \quad (9)$$

where we used the definition of the expected values together with Bayes' law $p(D_k|\theta) = \frac{p(\theta|D_k)p(D_k)}{p(\theta)}$. Next, we insert the Bayes estimator in Eq. (9) in order to write the Bayes risk solely as a function of the heuristic h which corresponds to Eq. (1) in the main text:

$$r[h|p(\theta)] = \mathbb{E}_{D_k} \left[\mathbb{E}_{\theta|D_k} \left(\|\mathbb{E}_{\theta|D_k}(\theta) - \theta\|_2^2 \right) \right] \quad (10)$$

$$= \mathbb{E}_{D_k} \left(\sum_j \text{Var}_{\theta_j|D_k} [\theta_j] \right) \quad (11)$$

$$= \mathbb{E}_{D_k} [\text{tr}(\text{Cov}_{\theta|D_k}[\theta])]. \quad (12)$$

The Bayes risk in the presence of limited resources

As discussed in the main text, the available resources for experiment designs may lead to episodes with different numbers of experiments. A Bayes risk which corresponds to the situation that the available resources are exhausted can easily be generalized from Eq.(1) in the main text as

$$r[h, p(\theta)] = \mathbb{E}_{D_{\text{end}}} (\text{tr}[\text{Cov}_{\theta|D_{\text{end}}}(\theta)]), \quad (13)$$

where D_{end} denotes all measurement outcomes for an episode. This means, the expectation $\mathbb{E}_{D_{\text{end}}}$ must be taken with respect to all data sets D_k which are compatible with the available resources.

Numerical Approximation of the Bayes risk

Let us first consider how to approximate Eq. (10) which is relevant for the experiment-limited case considered in our model study. The Bayes risk in Eq. (10) involves expected values over $p(\theta|D_k)$ and $p(D_k)$. The posterior $p(\theta|D_k)$ is represented in the SMC framework by a particle filter with weights w_j and particle locations $\theta_j \in \Theta$ which allows us to approximate the expected value of a function $f(\theta)$ as $E_{\theta|D_k}[f(\theta)] = \sum_j w_j f(\theta_j)$. The same approximation is used to compute the reward. Note that we set

$$\text{tr}(\text{Cov}_{\theta}[\theta]) = \text{const} \quad (14)$$

to a constant numerical value in order to avoid fluctuations of the reward originating from numerical uncertainties in the particle filter representing the prior $p(\theta)$.

The expected value over $p(D_k)$ is approximated as a sample mean: We sample 10^4 episodes with the RL environment. To each episode corresponds a series of measurement outcomes D_k and we can calculate $\text{tr}[\text{Cov}_{\theta|D_k}(\theta)]$. Note that the properties of the RL environment ensure that an episode with data D_k is sampled with probability $p(D_k)$. Then, we can approximate the expected value over $p(D_k)$ as the mean $\frac{1}{10^4} \sum_{D_k} \text{tr}[\text{Cov}_{\theta|D_k}(\theta)]$, where the sum runs over all 10^4 sampled series of measurement outcomes D_k .

The time-limited case we consider in our model study is an example where episodes can have different numbers of experiments. Moreover, we defined the time-limited case such that an episode ends when the agent has consumed more time than the given time limit. This means with the last experiment the RL agent (as well as the other heuristics) can go beyond the time limit. For the comparison of heuristics, this is not relevant mainly because all other experiment designs must lie within the time limit.

For the time-limited case, it would be insightful to have a time-resolved Bayes risk in analogy to the experiment-limited case, where for every number of experiments, we can compute a Bayes risk. Fig. 4 illustrates how we approximate such a time-resolved Bayes risk (actually, we average over 10^4 episodes and use 200 equally spaced times for interpolation). This approximation of the time-resolved Bayes risk is used for the time-limited cases shown in Fig. 2 in the main text.

Details on the training of the neural networks

Our results were obtained with NumPy 1.17.3 [49], QInfer 1.0a1 [26], Gym 0.14.0 [50], PyTorch 1.3.1 [51], Stable Baselines 2.9.0 [40], and Tensorflow 1.14.0 [52] libraries for Python 3.6.7. CEM and TRPO use fully connected neural networks (CEM: one hidden layer with

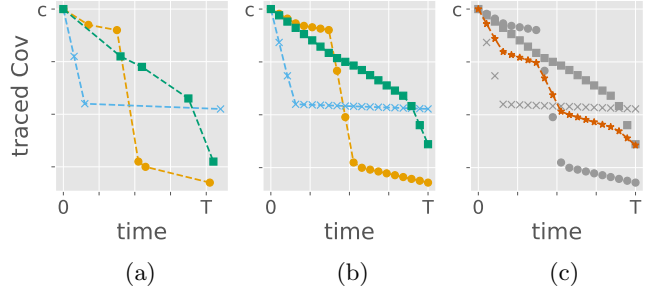


FIG. 4: Illustration of Bayes risk calculation for time-limited environments. Panel (a) shows 3 time series each of which corresponds to one episode (i.e., one Bayesian estimation): the “0th” data point at time 0 corresponds to the uncertainty c in the prior, see Eq. (14), and the k th data point in each time series corresponds to the k th experiment of that episode; its x coordinate is the accumulated time used for the first k experiments, and its y coordinate is the traced covariance $\text{tr}[\text{Cov}_{\theta|D_k}(\theta)]$. The time limit is T which is overstepped only with the last experiment of each episode. Linear interpolation of each time series (dashed lines) and evaluation of the linear interpolants at equally spaced times spanning $[0, T]$ yields panel (b), depicted with 20 equally spaced times. Then, we can take a time-wise mean of the data, i.e., for each of the 20 equally spaced times, we calculate the mean over all time series. This gives us the mean of the interpolated traced covariance [depicted as red stars in Panel (c)] which serves as an approximate time-resolved Bayes risk.

16 neurons, TRPO: two hidden layers with 64 neurons each).

The combination of CEM and TRPO

While CEM takes into account only the accumulated reward of full episodes, RL uses all the training data consisting of actions with corresponding observations and rewards. This allows TRPO to take into account the performance of single actions in order to improve the overall performance (at the end of episodes). If we pre-train TRPO with a CEM-trained NN (as a known heuristic), we obtain a purely machine learning based method for finding strong NN heuristics. The advantage of this two-step procedure over using only one of the algorithms is that we can use CEM for exploring the policy space (for our RL environments, CEM has proven to be better than TRPO in this respect) and TRPO for optimizing the heuristic further. A further speed-up and possibly an improvement of this method could be achieved by passing the neural network directly from CEM to TRPO (avoiding imitation learning). However, in the current

implementation, this is not possible because the neural networks used for CEM and TRPO are of different shape and not compatible.

Cross-entropy method

The input layer of the NN is defined by the observation. The output layer is determined by the number of actions (one action: time) and we choose 16 neurons in the hidden layer. The layers are fully connected. The hidden layer has the rectified linear unit (ReLU) as its activation function and the output layer has the softmax function as its activation function [53].

A schematic representation of the algorithm is given in Fig. 1(d) in the main text. The weights of the neural network form a vector $\mathbf{x}$. A generation is sampled from a Gaussian distribution $\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ with mean $\boldsymbol{\mu}$ and covariance $\Sigma = \mathbb{I}/2$. Before the first iteration, we sample $\boldsymbol{\mu}$ from $\mathcal{N}(\mathbf{0}, \Sigma)$. A generation consists of $K = 100$ individuals (=NNs). By running one episode (interacting with the RL environment) with each NN, we determine the $K_\epsilon = 10$ fittest individuals ($\mathbf{x}_1, \dots, \mathbf{x}_{K_\epsilon}$) as those with the largest reward accumulated over an episode. This concludes one iteration, the next generation is sampled from the distribution $\mathcal{N}(\boldsymbol{\mu}_{\text{new}}, \Sigma)$ with $\boldsymbol{\mu}_{\text{new}} = \frac{1}{K_\epsilon} \sum_{j=1}^{K_\epsilon} \mathbf{x}_j$. We run CEM for $N = 1000$ iterations. The final solution is given by $\boldsymbol{\mu}_{\text{new}}$ calculated in the last iteration. CEM is used to find 5 NNs for each RL environment and we plot results in Fig. 2 only for the NN heuristic which achieves the smallest Bayes risk after the maximum time or the maximal number of measurements.

Trust region policy optimization

We use TRPO [44] with the default multilayer perceptron policy as implemented in Stable Baselines 2.9.0 [40]. Hyperparameters which differ from their default values are $\gamma = 1$, $\lambda = 0.92$ and $\text{vf_stepsize} = 0.0044$. These hyperparameters were found to yield good results during initial testing of the algorithm, without doing a rigorous hyperparameter tuning. In particular, we do not use discounted rewards by setting the discount factor $\gamma = 1$, because even without reward damping the rewards are defined such that they tend to decay exponentially with the number of experiments. Further, smaller values for λ such as $\lambda = 0.92$ often work well together with large γ .

Pretraining is also implemented in Stable Baselines 2.9.0 [40] using the Adam optimizer [54]. Deviations from the default parameters [40] are the following: we use 10^4 (10^3) episodes, sampled from a known heuristic, as an expert dataset. Pretraining runs with 10^4 (10^3) epochs (the number of training iterations on the expert dataset) and a batch size for the expert dataset of 100 (10) (bracketed

values are used for the time-limited (ω, T_2^{-1}) estimation to speed up the computation).

Training runs for 500 iterations. However, we use an exit condition which can stop the training earlier. The exit conditions stops training if the policy entropy drops below 0.005 [40].

TRPO is pretrained once per heuristics (PGH, σ^{-1} , the best of five CEM-trained NNs), and then trained 5 times for each of the pretrainings, and we plot the results in Fig. 2 only for the best NN heuristic, i.e., with the largest Bayes risk at the end of the episodes.

SUPPLEMENTARY INFORMATION

Supplementary Note 1: Comparison of experiment designs from different heuristics

Here we provide plots of the distribution of experiment designs, i.e., experiment times, chosen by the four adaptive heuristics discussed in the main text: the particle-guess heuristic (PGH), the σ^{-1} heuristic, a neural network trained with the cross-entropy method (CEM), and a neural network trained with trust region policy optimization (TRPO). The plotted experiment designs correspond precisely to the data used to calculate the Bayes risk in the main text (see Fig. 2 in the main text). Data consist for each estimation problem and each heuristic of 10^4 episodes. Note that the distribution of experiment designs shown in Supplementary Figs 5 and 6 do not reveal the adaptivity of experiment designs. Instead we can see the frequency of experiments designs for different experiments, i.e., which times are chosen for the first experiment of each episode (Bayesian estimation), which for the second experiment, and so on. The diversity we find for different experiment-design heuristics is remarkable. Also the diversity between different environments underpins that experiment-design heuristics should be tuned to the particular estimation problem.

Table I shows the limits on the number of experiments and the available time for all estimation problems. Note that for numerical convenience, time-limited (experiment-limited) problems also have a limit on the number of measurements (the available time). In particular, for time-limited problems a limit on the number of experiments helps to avoid that a RL agent chooses very many small experiment times which would lead to very long episodes, i.e., to very long run times. Also from a physical perspective, small experiment times are not sensible due to overheads for readout and preparation. Such overheads can be taken into account as a dead time of the sensor between experiments which can be easily implemented as a part of the RL environments. In case of experiment-limited environments, the time limit 10^{27} avoids issues with large numerical values and it is chosen large enough for the exp-sparse heuristic with 500 experiments, which chooses exponentially increasing times.

estimation problem	experiment limit	time limit
ω estimation, $T_2 = \infty$, experiment-limited	20	10^{27}
ω estimation, $T_2 = \infty$, time-limited	100	100
ω estimation, $T_2 = 100$, experiment-limited	125	10^{27}
ω estimation, $T_2 = 100$, time-limited	1000	2500
ω estimation, $T_2 = 10$, experiment-limited	125	10^{27}
ω estimation, $T_2 = 10$, time-limited	1000	2500
(ω, T_2^{-1}) estimation, experiment-limited	500	10^{27}
(ω, T_2^{-1}) estimation, time-limited	4000	2500

TABLE I: Limits on the number of experiments and the available time for all estimation problems considered in this work.

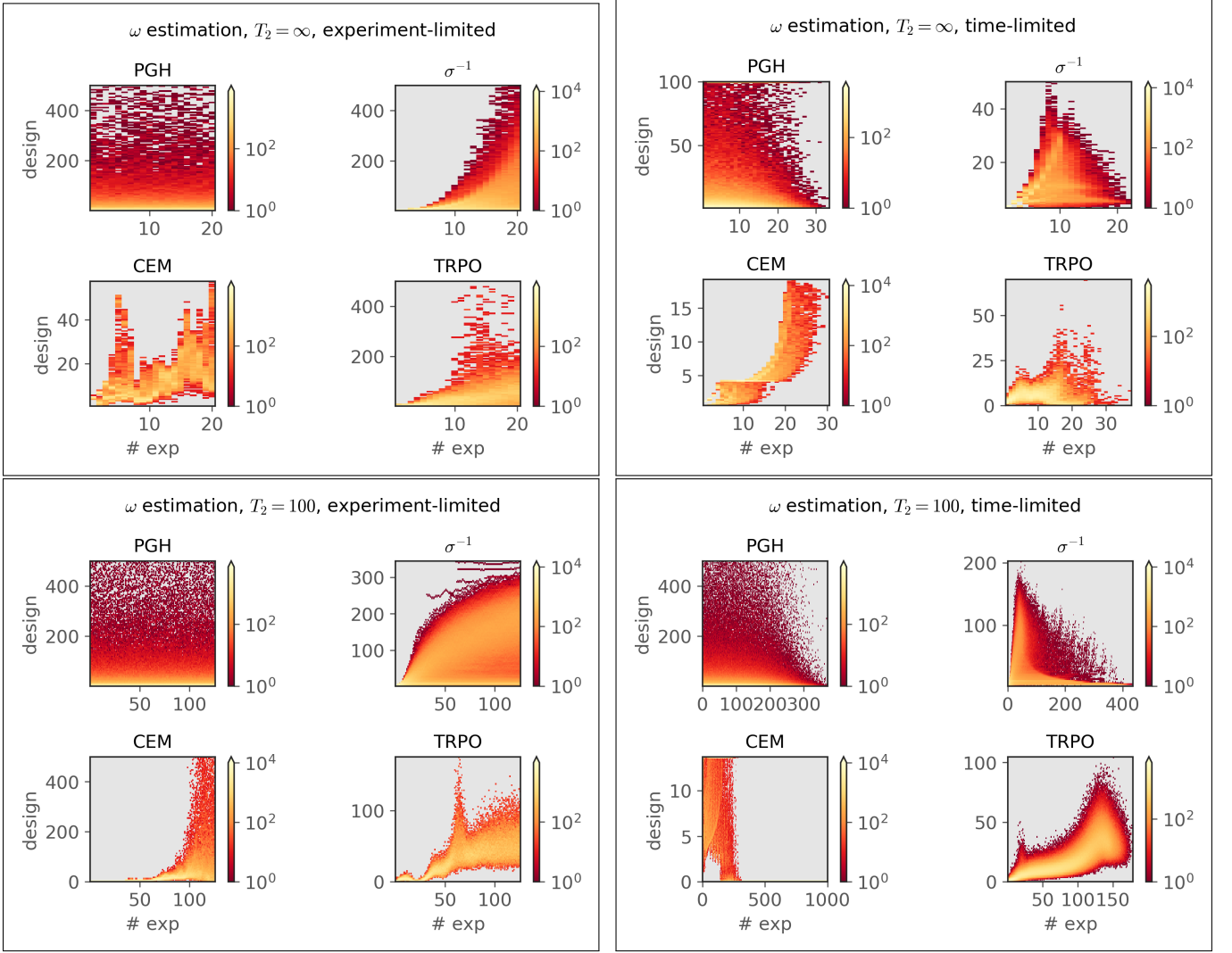


FIG. 5: Distribution of experiment designs (y axis). On the x axis we show the experiment number, i.e., $x = 10$ corresponds to the 10th experiment in each episode. For the sake of readability, we cut off y data at $t = 500$. Note that in the time-limited cases, CEM reaches sometimes the numerical threshold for the number of experiments by choosing many experiments with very small t . This manifests itself in the corresponding plots as a bright line close to $t = 0$ (see for example the case of ω estimation, $T_2 = 100$, time-limited). If the gray background of the plots is visible, there are no experiment designs for the corresponding values.

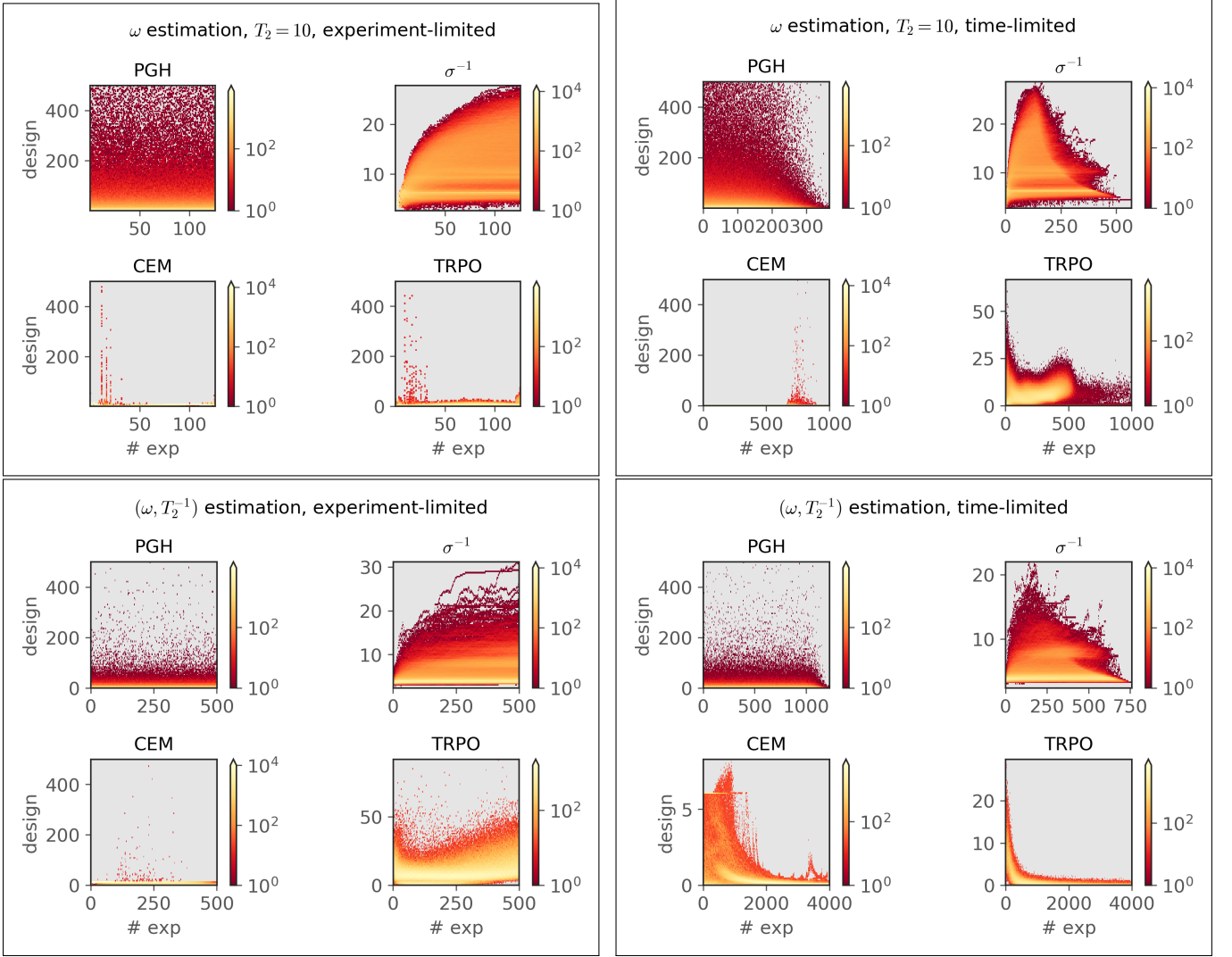


FIG. 6: See caption of Fig. 5, the only difference is that this figure shows data for other estimation problems (see the figure titles).